

Trabajo Practico 1: Ingeniería de Prompts y Git/Github

Actividad 1: Calibración del Modelo (Temperatura)

1.a

Mensaje de Prompt:

“Necesito que realices un programa simple el cual pueda construir tablas de verdad en Python, basados en la lógica proposicional

Temperatura: 0 “

Este programa trabaja con las proposiciones p y q y permite generar tablas para:

- Negación: $\neg p$
- Conjunción: $p \wedge q$
- Disyunción: $p \vee q$
- Implicación: $p \rightarrow q$
- Bicondicional: $p \leftrightarrow q$

 Python



 Ejecutar

```
print("=== CONSTRUCTOR DE TABLAS DE VERDAD ===")
```

```
print("1. Negación:  $\neg p$ ")
print("2. Conjunción:  $p \wedge q$ ")
print("3. Disyunción:  $p \vee q$ ")
print("4. Implicación:  $p \rightarrow q$ ")
print("5. Bicondicional:  $p \leftrightarrow q$ ")
```

```
opcion = input("Seleccione una opción: ")
```

```
if opcion == "1":
```

```
    print()
    print("p |  $\neg p$ ")
    print("-----")
```

```
    p = True
    print(p, "|", not p)
```

```
    p = False
    print(p, "|", not p)
```

```
elif opcion == "2":
```

```
    print()
    print("p | q |  $p \wedge q$ ")
    print("-----")
```

```
Python 📄 Ejecutar

for p in [True, False]:
    for q in [True, False]:
        resultado = p and q
        print(p, "|", q, "|", resultado)

elif opcion == "3":
    print()
    print("p    | q    | p v q")
    print("-----")

    for p in [True, False]:
        for q in [True, False]:
            resultado = p or q
            print(p, "|", q, "|", resultado)

elif opcion == "4":
    print()
    print("p    | q    | p → q")
    print("-----")

    for p in [True, False]:
        for q in [True, False]:

            resultado = (not p) or q

            print(p, "|", q, "|", resultado)

elif opcion == "5":
    print()
    print("p    | q    | p ↔ q")
    print("-----")

    for p in [True, False]:
        for q in [True, False]:

            resultado = p == q

            print(p, "|", q, "|", resultado)

else:
    print("Opción incorrecta.")
```

1.b

Mensaje de Prompt:

“Ahora necesito que me des una idea de cómo podría hacer un programa, el cual cree tablas de la verdad

Temperatura: 1“

Una buena idea es hacer un programa tipo “**Generador de Tablas de Verdad**”, donde el usuario escriba una proposición lógica y el programa genere automáticamente todas las combinaciones posibles de valores de verdad.

Por ejemplo, el usuario podría ingresar:

p and q



y el programa debería mostrar:

p	q	Resultado
True	True	True
True	False	False
False	True	False
False	False	False



Para construirlo de forma simple, yo lo dividiría en 4 etapas:

1. Pedir las proposiciones

Por ejemplo:

`</>` Python



Ejecutar

```
cantidad = int(input("¿Cuántas proposiciones tendrá la tabla? "))
```

Si el usuario elige **2**, trabajás con:

p
q



Si elige **3**:

p
q
r



2. Generar todas las combinaciones posibles

Con dos proposiciones necesitas:

```
V V  
V F  
F V  
F F
```



Una forma sencilla, si solamente trabajan con `p` y `q`, es usar dos `for`:

`</>` Python



▶ Ejecutar

```
for p in [True, False]:  
    for q in [True, False]:  
        print(p, q)
```

Eso automáticamente genera las cuatro filas.

3. Pedir qué operación quiere realizar

Podrías mostrar un menú:

```
1. p ^ q  
2. p v q  
3. p -> q  
4. p <= q  
5. ~p
```



El usuario elige una opción.

Después calculás el resultado dependiendo de lo elegido.

Por ejemplo:

`</>` Python



▶ Ejecutar

```
if opcion == "1":  
    resultado = p and q
```



Para OR:

```
</> Python
resultado = p or q
```



Ejecutar

Para implicación:

```
</> Python
resultado = (not p) or q
```



Ejecutar

4. Mostrar la tabla

Tu estructura general podría ser algo así:

```
</> Python
print("GENERADOR DE TABLAS DE VERDAD")

print("1. Conjunción")
print("2. Disyunción")
print("3. Implicación")
print("4. Bicondicional")

opcion = input("Ingrese una opción: ")

for p in [True, False]:
    for q in [True, False]:

        # Acá calculás el resultado

    print(p, q, resultado)
```



Ejecutar

Actividad 2: Restricción de Longitud (Max Length)

Mensaje:

“Construye una tabla de la verdad en Python, basada en la logica proposicional.

Max. Length: 20”

Construye una tabla de la verdad en Python, basada en la logica proposicional
Max. Length: 20

```
</> Python
print("p\tq\t p AND q")

for p in [True, False]:
    for q in [True, False]:
        print(p, q, p and q, sep="\t")
```



Ejecutar



Actividad 3: Secuencias de Parada (Stop Sequences)

Mensaje:

“Construye una tabla de la verdad en Python, basada en la logica proposicional.

Stop Sequences: conditional not(p and q) ”

Construye una tabla de la verdad en Python, basada en la logica proposicional
Stop Sequences: conditional not(p and q)

```
Python
print("p\tq\t~(p ^ q)")

for p in [True, False]:
    for q in [True, False]:
        resultado = not (p and q)
        print(p, q, resultado, sep="\t")
```

 Ejecutar

    ...

Actividad 4: Control de Repetición y Novedad (Penalizaciones)

4.a

Mensaje:

“Construye una tabla de la verdad en Python, basada en la logica proposicional.

Frecuency Penalty

Construye una tabla de la verdad en Python, basada en la logica proposicional
Frecuency Penalty

```
Python
print("p\tq\tp OR q")

for p in [True, False]:
    for q in [True, False]:
        resultado = p or q
        print(p, q, resultado, sep="\t")
```

 Ejecutar

4.b

Mensaje:

“Construye una tabla de la verdad en Python, basada en la logica proposicional.

Presence Penalty

Construye una tabla de la verdad en Python, basada en la logica proposicional Presence Penalty

Python

```
print("p\tq\tp → q")

for p in [True, False]:
    for q in [True, False]:
        resultado = (not p) or q
        print(p, q, resultado, sep="\t")
```

Ejecutar

Actividad 5: Reglas de Sintaxis Avanzada

Prompt:

Lógica Proposicional 2108.docx
Documento

Quiero que construyas una tabla de verdad basada en la lógica proposicional. Hazlo de forma eficiente y con buenas practicas de programación

Sos un profesor de matemática con 5 años de experiencia dando clases que además sabe programar.

""Guíate por el documento que te adjunto: ""

Formato: Documento de Python

Resultado:

Preparé el programa en formato `.py`, siguiendo las operaciones del documento: negación, conjunción, disyunción, condicional e bicondicional. En particular, el código usa la equivalencia $p \rightarrow q \equiv \neg p \vee q$ y la definición del bicondicional que aparecen en el material. [Lógica Proposicional 2108](#)

También mantiene la notación `V/F`, como en las tablas del documento, y genera automáticamente las cuatro combinaciones posibles de `p` y `q`.

[Descargar tabla_verdad.py](#)

La salida queda conceptualmente así:

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

El programa evita `eval`, separa cada operación lógica en funciones claras y usa `itertools.product` para generar las combinaciones de valores de verdad de forma limpia y escalable. [Código](#)

[tabla_verdad.py](#)
Código

Fuentes

Contenido del archivo generado:

```
"""
```

Tabla de verdad basada en lógica proposicional.

Incluye:

- Negación: $\neg p$
- Conjunción: $p \wedge q$
- Disyunción: $p \vee q$
- Condicional: $p \rightarrow q$
- Bicondicional: $p \leftrightarrow q$

```
"""
```

```
from itertools import product
```

```
def valor_logico(valor: bool) -> str:
```

```
    """Convierte True/False en V/F para mostrar la tabla."""
    return "V" if valor else "F"
```

```
def implicacion(p: bool, q: bool) -> bool:
```

```
    """ $p \rightarrow q$  equivale a  $\neg p \vee q$ ."""
```

```
return (not p) or q
```

```
def bicondicional(p: bool, q: bool) -> bool:  
    """p ↔ q es verdadero cuando p y q tienen el mismo valor lógico."""  
    return p == q
```

```
def mostrar_tabla() -> None:  
    """Construye y muestra la tabla de verdad para p y q."""  
    encabezados = ["p", "q", "¬p", "p ∧ q", "p ∨ q", "p → q", "p ↔ q"]  
  
    print(" | ".join(f"{titulo:^7}" for titulo in encabezados))  
    print("-" * 73)
```

```
for p, q in product([True, False], repeat=2):  
    fila = [  
        valor_logico(p),  
        valor_logico(q),  
        valor_logico(not p),  
        valor_logico(p and q),  
        valor_logico(p or q),  
        valor_logico(implicacion(p, q)),  
        valor_logico(bicondicional(p, q)),  
    ]  
  
    print(" | ".join(f"{valor:^7}" for valor in fila))
```

```
if __name__ == "__main__":  
    mostrar_tabla()
```